COMPLETE API & POSTMAN NOTES

Beginner-friendly concepts • Restful Booker hands-on lab • Real-world QA scenarios

Includes explanations, sample requests, test scripts, common mistakes, and best practices.

Prepared from the supplied Theory.txt and Flow(4).txt learning materials, with clarifications and practical examples.

1. Learning Objectives

After studying these notes, a beginner should be able to explain APIs, understand the client-server request/response cycle, distinguish SOAP from REST, use Postman workspaces and collections, create and retrieve bookings, authenticate, update and delete records, interpret HTTP status codes, and apply practical API-testing best practices.

2. What Is an API?

API stands for Application Programming Interface. It is a defined way for software components to communicate. An API describes which operations are available, what information a caller must send, and what response it can expect.

Example: A restaurant menu is like an API contract. It lists what you can order and the information needed. The kitchen is like the server; it processes the order and returns the result through the waiter.

Example: A travel website needs booking information. It sends a request to a booking service API; the service processes it and returns data such as the booking ID, guest name, price, and dates.

> ***Important note:*** *An API is not necessarily just a collection of functions. Web APIs commonly expose resources and operations over HTTP; APIs can also exist inside libraries, operating systems, and other software.*

3. How an API Works

• 1. Client: a browser, mobile app, Postman, or automated test sends a request.

• 2. Request: includes a URL/endpoint, HTTP method, optional headers, and sometimes a body.

• 3. Network/server: the request reaches the service.

• 4. Processing: the server validates the request, checks authentication/authorization, performs business logic, and may read or write a database.

• 5. Response: the server returns an HTTP status code, headers, and often a response body.

Example: Example flow: Postman sends GET /booking/1 → the booking service looks up booking 1 → it returns 200 OK and JSON if found, or an error response if the booking does not exist.

4. Common API Types

Private API

Used internally within an organization or system. Example: an order service calls an internal inventory service.

Public API

Made available to external developers or users. It may require registration, API keys, authentication, rate limits, or payment.

Partner API

Shared between approved business partners. Example: a retailer and a delivery provider exchange shipment details.

Composite API

Combines multiple operations or data sources into one call or workflow. This can reduce the number of calls a client must make.

5. Web Services: SOAP and REST

SOAP

SOAP stands for Simple Object Access Protocol. It is a messaging protocol with a structured envelope, commonly using XML. SOAP services often describe operations using WSDL and may use WS-* standards for security and reliability. SOAP can operate over HTTP and other transports.

REST

REST stands for Representational State Transfer. It is an architectural style, not a protocol. RESTful APIs commonly expose resources through URLs and use HTTP methods such as GET, POST, PUT, PATCH, and DELETE. JSON is common, but XML and other representations are possible.

Important note: *Correction to the source notes: REST does not mean "Representational Stateful APIs"; it means "Representational State Transfer." REST is commonly designed to be stateless between requests. Also, SOAP can use HTTP headers; it is incorrect to say SOAP has no headers. SOAP messages are typically XML, while REST responses can use JSON, XML, text, or other media types.*

Aspect	SOAP	REST
Type	SOAP is a protocol with a formal message envelope	REST is an architectural style
Typical format	SOAP commonly uses XML envelopes	REST commonly uses JSON but is not limited to it
Contract/documentation	SOAP contracts often use WSDL	REST APIs commonly document endpoints with OpenAPI or similar specifications

6. HTTP Methods and CRUD

Item	Details
Method	Typical purpose
Example	Notes
GET	Read/retrieve data
GET /booking/1	Should not change server state
POST	Create a resource or submit an operation
POST /booking	Often returns a new ID
PUT	Replace/update a resource representation
PUT /booking/634	Usually sends the full resource representation
PATCH	Partially update a resource
PATCH /booking/634	Sends only changed fields when supported
DELETE	Delete a resource
DELETE /booking/634	Removes the target resource when authorized

CRUD means Create, Read, Update, Delete. A common mapping is POST, GET, PUT/PATCH, and DELETE respectively. The exact behavior depends on the API contract.

7. HTTP Request Components

Endpoint / URL

The address of the API resource, for example https://restful-booker.herokuapp.com/booking. A path parameter identifies a particular resource, such as /booking/1.

Method

The operation to perform: GET, POST, PUT, PATCH, DELETE, etc.

Headers

Metadata sent with a request. Content-Type describes the request body format (for example application/json). Accept describes response formats the client can accept. Authorization or Cookie may carry credentials according to the API's authentication design.

Request body

Data sent to the server, commonly with POST, PUT, or PATCH. JSON example: { "firstname": "Asha", "lastname": "Rao" }. GET requests typically do not need a body.

Response

Usually includes a status code, response headers, and body. Postman also displays response time and response size.

8. HTTP Status Codes to Know

Item	Details
Code / class	Meaning
Typical interpretation	200 OK
Request succeeded	Common for successful GET or update
201 Created	Resource created
Common for successful POST creation	202 Accepted
Accepted for processing	Work may complete asynchronously
204 No Content	Succeeded with no response body
Common for DELETE	301/302
Redirect	Client may be redirected
400 Bad Request	Invalid request
Malformed JSON or validation error	401 Unauthorized
Authentication missing/invalid	Credentials or token may be wrong
403 Forbidden	Not permitted
Identity may be known but lacks permission	404 Not Found
Resource/route not found	Wrong ID or endpoint
409 Conflict	Conflict with current state
Duplicate or version conflict	415 Unsupported Media Type
Unsupported body format	Check Content-Type
422 Unprocessable Content	Content understood but invalid
Business validation failed	429 Too Many Requests
Rate limit exceeded	Reduce request rate
500 Internal Server Error	Server failed unexpectedly
Investigate server logs	502/503/504
Gateway/service unavailable/timeout	Dependency or infrastructure issue

Important note: *A 2xx status usually indicates success, but always validate the response body and business outcome too. A request can return 200 with incorrect data, so status alone is not a complete test.*

9. Why Use Postman Workspaces and Collections?

Workspace

A workspace is a place to organize collaboration, collections, environments, and related API work. The exact workspace types and sharing controls can change with Postman's interface and plan.

Collection

A collection stores related requests. Example: a Booking API collection can contain Positive Endpoints and Negative Scenarios folders.

Why collections help

• Keep related API requests organized.

• Reuse authentication, variables, and configuration.

• Run request sequences with the Collection Runner.

• Share a repeatable test set with teammates.

Example: Team scenario: a QA engineer groups all payment requests in a Payment API collection. A teammate can quickly find and run the relevant tests rather than searching through dozens of unsaved requests.

> ***Important note:*** *Saving requests in a collection does not automatically pass data between requests or guarantee dependencies. Use variables and scripts to capture IDs/tokens and configure execution order.*

10. Practical Lab: Booking API in Postman

The source lesson uses the public demo API Restful Booker. API documentation: https://restful-booker.herokuapp.com/apidoc/index.html. Public demo services may be reset, rate-limited, or temporarily unavailable, so treat returned IDs and results as examples rather than permanent data.

Step 1 — Create a workspace

• 1. Open Postman and choose Workspaces → Create Workspace.

• 2. Enter a name such as API Testing Practice.

• 3. Choose the appropriate visibility/type. Use a private workspace for work containing secrets or sensitive data; do not publish credentials.

Step 2 — Create a collection and folders

• 1. Create a collection named Booking API.

• 2. Add folders named Positive Endpoints and Negative Scenarios.

• 3. Save each request into the appropriate folder.

Step 3 — GET all booking IDs

Method: GET | URL: https://restful-booker.herokuapp.com/booking

Save as: GET Booking IDs. Click Send. The endpoint normally returns a list of booking identifiers. Inspect status, response time, response size, and body. Do not assume that a particular ID will always exist.

Step 4 — GET one booking by ID

Method: GET | URL: https://restful-booker.herokuapp.com/booking/1

Save as: GET Booking Details by ID. A successful response can include firstname, lastname, totalprice, depositpaid, bookingdates (checkin and checkout), and additionalneeds. If ID 1 is unavailable in the demo environment, use an ID returned by the list endpoint.

Step 5 — POST to create a booking

Method: POST | URL: https://restful-booker.herokuapp.com/booking

Header: Content-Type: application/json. Select Body → raw → JSON and send this example:

```
{"firstname":"Asha","lastname":"Rao","totalprice":250,"depositpaid":true,"bookingdates":{"checkin":"2026-12-01","check
```

A successful creation normally returns a bookingid and a booking object. Copy the actual bookingid from your response; do not hard-code the sample ID 634 from a training video because IDs can differ.

Step 6 — Verify the created booking

Send GET https://restful-booker.herokuapp.com/booking/{{bookingId}} using the ID from the POST response. Confirm that the returned guest name, dates, and other fields match the request. This is an example of a post-condition check.

Step 7 — Generate an authentication token

Method: POST | URL: https://restful-booker.herokuapp.com/auth. Header: Content-Type: application/json. Body:

```
{"username":"admin","password":"password123"}
```

For this public demo, the commonly documented sample credentials above return a token when accepted. A demo credential is not appropriate for a real production service. Store real credentials in Postman variables/secrets, not in shared request bodies.

Step 8 — PUT to update a booking

Method: PUT | URL: https://restful-booker.herokuapp.com/booking/{{bookingId}}. Send Content-Type: application/json and the authentication cookie in the format required by the API, commonly Cookie: token={{token}}. Use the full booking representation, for example:

```
{"firstname":"Asha","lastname":"Kulkarni","totalprice":275,"depositpaid":true,"bookingdates":{"checkin":"2026-12-01","
```

PUT generally replaces the full representation, so include all required fields. Check the API documentation for required fields and accepted authentication format. A successful response should be followed by GET to verify the update.

Step 9 — DELETE a booking

Method: DELETE | URL: https://restful-booker.herokuapp.com/booking/{{bookingId}}. Send the required authentication cookie. Execute DELETE only against a test booking you created or are explicitly allowed to remove. Then send GET for the same ID and confirm the expected not-found behavior.

11. Use Variables Instead of Hard-Coding IDs and Tokens

Variables make requests reusable across environments and test runs. Create collection variables or environment variables such as baseUrl, bookingId, and token.

```
GET {{baseUrl}}/booking/{{bookingId}}

Example baseUrl value:
https://restful-booker.herokuapp.com
```

After a POST request, open the Scripts tab and use a post-response script similar to the following to capture the returned ID. Postman script APIs can change; this uses the current pm-style interface commonly supported by Postman.

```
const body = pm.response.json();
pm.test("Booking was created", function () {
  pm.expect(pm.response.code).to.be.oneOf([200, 201]);
  pm.expect(body).to.have.property("bookingid");
});

pm.collectionVariables.set("bookingId", String(body.bookingid));
```

For the token request, capture the token from the response according to the response structure returned by the service. Do not assume every API uses the same field name or authentication scheme.

12. Negative Scenarios and Validation

Negative testing checks how the API behaves when inputs, authentication, or resource identifiers are invalid. Put these requests in the Negative Scenarios folder.

- GET a booking using a nonexistent ID; verify the documented not-found response.
- POST malformed JSON; expect a client error rather than a successful creation.
- Omit a required field; verify the validation response.
- Send an unsupported Content-Type; verify whether the API returns 415 or another documented error.
- Try PUT/DELETE without a token or with an invalid token; verify authentication enforcement.
- Send an invalid date range (checkout before checkin); verify business validation if the API supports it.

Important note: *Do not assert one universal status code for every negative case. Assert the status and error schema documented by the specific API. Avoid destructive negative tests on shared or production systems.*

13. Example Postman Tests

In a request's Scripts → Post-response section, add checks to validate the status and response shape. Example for GET Booking IDs:

```
pm.test("Status is 200", function () {
  pm.response.to.have.status(200);
});

pm.test("Response is an array", function () {
  const body = pm.response.json();
  pm.expect(body).to.be.an("array");
});
```

Example for GET booking details:

```
pm.test("Booking contains expected fields", function () {
  const body = pm.response.json();
  pm.expect(body).to.have.property("firstname");
  pm.expect(body).to.have.property("lastname");
  pm.expect(body).to.have.property("bookingdates");
});
```

Tests should validate meaningful behavior, not just that the request did not fail. Avoid overly brittle assertions on data that legitimately changes.

14. Real-Time QA Scenarios

Scenario A — Booking creation flow

Create a booking with POST, capture the booking ID, retrieve it with GET, compare key fields, update it with PUT, retrieve it again, then delete the test record. This verifies the lifecycle of a resource.

Scenario B — Authentication

Call a protected endpoint without a token, with an invalid token, and with a valid token. Confirm that the service rejects unauthorized requests and permits authorized requests. Do not expose real tokens in screenshots, shared collections, or logs.

Scenario C — Data validation

Try missing guest names, invalid dates, negative prices, wrong data types, and malformed JSON. Check the status code and response body against the API contract.

Scenario D — Performance smoke check

Record response times for repeated requests under consistent conditions. A single response-time reading is not enough to establish performance. Use a load-testing tool and an agreed threshold for meaningful performance testing.

Scenario E — Regression collection

Keep stable positive and negative tests in a collection. Run them after a new build or deployment, and use environments for development, QA, and staging URLs.

15. Best Practices

• 1. Read the API documentation before building requests.

• 2. Use meaningful request names, folders, and descriptions.

• 3. Use variables for base URLs, IDs, and tokens; avoid hard-coded dynamic values.

• 4. Never commit or publish passwords, API keys, or tokens. Use Postman's secret/variable controls and restrict sharing.

• 5. Keep destructive requests such as DELETE clearly labeled and use test data.

• 6. Validate status code, response body, headers, and business rules.

• 7. Add negative tests, boundary values, and malformed-input tests.

• 8. Keep environment-specific configuration separate from requests.

• 9. Do not assume IDs or demo data are permanent.

• 10. Avoid using public demo endpoints for confidential or production data.

• 11. For automation, make tests repeatable and independent where practical.

• 12. Document expected behavior and make assertions match the API contract.

16. Common Beginner Mistakes

Item	Details
Mistake	Why it is a problem
Better approach	Using GET for a write operation
Violates expected HTTP semantics	Use the documented method
Forgetting Content-Type	Server may not parse the body correctly
Set application/json for JSON bodies	Sending invalid JSON
Request may fail validation or parsing	Use valid JSON and check syntax
Hard-coding booking ID	IDs can differ between runs
Capture the ID from POST response	Using a fake token value
Protected request will fail	Capture the real token response and use the documented cookie/header
Checking only status code	Response can still contain wrong data
Assert key fields and business rules	Sending partial body with PUT
Required fields may be overwritten/missing	Send full representation or use PATCH if supported
Deleting shared data	Can disrupt other learners

17. Quick Revision / Interview Questions

• 1. What does API stand for, and what problem does it solve?

• 2. Explain the client-server request/response cycle.

• 3. What is the difference between an API, a web service, and an endpoint?

• 4. How do SOAP and REST differ?

• 5. Explain GET, POST, PUT, PATCH, and DELETE.

• 6. What are headers and why is Content-Type important?

• 7. What is the difference between 401 and 403?

• 8. Why should a Postman collection use variables?

• 9. How would you verify that a POST request created a resource correctly?

• 10. How would you test an API when the booking ID does not exist?

• 11. How do you keep tokens and passwords safe in Postman?

• 12. Why is a 200 response not sufficient proof that a test passed?

Suggested answers: Explain the concept first, then give a concrete example. For scenario questions, describe the request, expected status/body, assertions, and cleanup.

18. Final Practice Exercise

Build a Booking API collection with folders for positive and negative cases. Implement GET all IDs, GET by ID, POST create, POST auth, PUT update, and DELETE. Use variables for baseUrl, bookingId, and token. Add assertions for status codes and response fields. Run the flow in order, verify each change with GET, and clean up only the test booking you created.

Completion checklist: [ ] Collection is organized. [ ] Requests are saved with clear names. [ ] JSON bodies are valid. [ ] Dynamic ID is captured from POST. [ ] Token is captured and used correctly. [ ] Positive tests validate response data. [ ] Negative tests validate documented errors. [ ] Secrets are not exposed. [ ] Test data is cleaned up safely.

19. Source Coverage and Notes

These notes are based on the two supplied files: Theory.txt (API fundamentals, API categories, SOAP/REST, and collections) and Flow(4).txt (Postman workspace/collection setup and the Restful Booker GET, POST, authentication, PUT, and DELETE walkthrough). Explanations, testing scripts, practical scenarios, and best practices have been added for beginner learning.

> ***Important note:*** *Important: Postman interface labels, demo API behavior, credentials, and response examples can change. When a response differs from this guide, check the current API documentation and the actual response. The guide is educational, not a guarantee that a public demo service will always be available.*